

Portfolio

Danila Nelin · fullstack TypeScript / Vue / NestJS · Solana, TON

[t.me/spaster](#) [nelindanila7@gmail.com](#) [github.com/spasster](#) [sol-meme.com](#) [solpump.gg](#) [@polyxcopy_bot](#)

Four pieces of work: sol-meme.com, solpump.gg (spun out of sol-meme), Lotty from the PROD finals, and PolyX.

SKILLS

Scale: basic → advanced → confident → professional.

SKILL	LEVEL	EVIDENCE
TypeScript fullstack (Vue 3, NestJS)	confident	sol-meme.com and solpump.gg, I wrote the code
Data model and money	advanced	solpump: double-entry ledger, lock/settlement/withdraw, per-user deposit addresses, AES-256-GCM on keys
Realtime	advanced	Socket.IO prices and chat (solpump)
Queues and background jobs	advanced	BullMQ: deposits / options / sweep (solpump); Bull + Redis for notifications (Lotty); copy-engine workers (PolyX)
Observability and deploy	advanced	Grafana, Prometheus, Loki, Sentry; Docker Compose, Nginx; GitHub Actions → SSH (solpump)
Solana	confident	SPL / Raydium CPMM / Metaplex / IPFS; on-chain decode of PumpSwap, Meteora DLMM; burn/close ATA
TON	basic	TonConnect in a Mini App; Go service with tonutils-go + NATS; TON deposits in PolyX
Python backend	advanced	CLOB / web3.py (PolyX); aiogram + payments (CryptoPay, card)
Experiment analytics	advanced	Lotty: MongoDB + ClickHouse, decide/event/report/guardrail, k6, Jest
Pitching and working with a partner	advanced	PROD finals; Technostrelka — 1st place; PolyX — I am the developer, partner is idea author and CEO

CONTESTS AND HACKATHONS

- PROD finals: 65 (individual), 62 (team) — Lotty case.
- Technostrelka — 1st place, Neymark case.
- Regional product-programming hackathons (Ryazan): 2nd and 3rd place, sports rank.
- IT Quantum: bots, CV (mask detection), Android transit payment app.

SPL token launchpad and prediction on Solana

<https://sol-meme.com>

Task	User path: create a token → Raydium pool → referrals, plus prediction with SOL payouts.
Role	All of the code: API, frontend, Solana, bot, deploy, monitoring. Product and design were not me.
Stack	Vue 3, Pinia, Vite, NestJS, MongoDB (Mongoose), Redis, Socket.IO, @solana/web3.js, Raydium SDK v2, Metaplex, Pinata/IPFS, Docker, Nginx, Grafana, Prometheus, Loki, aiogram.
Solution	- Token create: upload image → build unsigned tx → confirm after wallet signature. Private keys never hit the backend. - Metadata goes to IPFS (Pinata), pool is Raydium CPMM. The service does not lock LP. - Referrals accrue only from token-create confirm and burn/close, not from a public POST. - Prediction: price/pool snapshot, Birdeye oracle, payouts in SOL. - Separate Telegram bot. Blog + sitemap. Dashboards for product and server metrics.
Result	About a year in production. Docker Compose, Mongo backups.
Link	sol-meme.com

Custodial UP/DOWN options on meme tokens

<https://solpump.gg>

Task	Spun prediction out of sol-meme: short UP/DOWN bets, funds on the service balance, price from a real pool, live chart, chat, deposit and withdraw.
Role	All of the code: ledger, oracles, realtime, CI/CD, monitoring.
Stack	Vue 3, NestJS, Prisma, PostgreSQL, Redis, BullMQ, Socket.IO, Sentry, Prometheus, GitHub Actions, Docker, Nginx.
Solution	- Double-entry ledger: deposit → lock → settlement → withdraw. Serializable transactions, per-user advisory lock. DB triggers block update/delete of entries. Each transaction must sum to zero. - Each user gets a custodial Solana deposit address. Key is AES-256-GCM encrypted, never logged. No memo: whatever lands on the address belongs to that user. - Watching every wallet live does not scale. Hot wallets get live <code>onAccountChange</code>. Dormant ones are scanned in round-robin batches. Opening the deposit page subscribes and enqueues a scan. - Price: Pump.fun bonding curve, else DexScreener for discovery only → then on-chain decode (PumpSwap / Raydium CPMM / Meteora DLMM). Refresh stays on that pool while it is alive. - Socket.IO namespaces <code>/options</code> and <code>/prices</code>. BullMQ: deposits, options, sweeps. Auth: JWT + refresh in an <code>HttpOnly</code> cookie, rate limit, Sentry. - CI on main: frontend type-check, backend <code>npm test</code>, image build, SSH deploy only if all of that is green.
Result	Split out of sol-meme. Markets without an account; trade and withdraw after login.
Link	solpump.gg

Split-testing platform — PROD finals

github.com/spasster/lotty-ab-platform

Task	Internal service: flags, experiments, “which variant to show”, event stream, reports, launch approval, auto-stop if a metric tanks.
Role	Backend and infra. PROD individual — 65, team — 62.
Stack	NestJS, MongoDB, ClickHouse, Redis, Bull, JWT, OpenAPI / Swagger, Pino, Prometheus, Grafana, Docker, Jest, k6.
Solution	• Deterministic variant assignment by subject + flag + attributes. decide responses cached in Redis. • Events (impressions, clicks, conversions, errors) go to ClickHouse, partitioned by month. MongoDB holds flags, experiments, users, learnings. • Guardrails stop the experiment on degradation. Telegram/email notifications go through Bull; the API does not wait on SMTP. • Lifecycle: create → review → approve → pause / finish. Targeting DSL with parser tests. • C4, runbook, 12 demo scripts, tests.
Result	Finals case. Compose, metrics, tests, public repo.
Link	github.com/spasster/lotty-ab-platform

Copy-trading on Polymarket

t.me/polyxcopy_bot

Task	Show top Polymarket traders, let users subscribe to copy trades via Telegram, take a deposit and execute mirrored orders.
Role	Developer: web, bot, workers, wallets, bridge, deploy. Partner is idea author and CEO.
Stack	Nuxt 4, Pinia, Tailwind, shadcn-vue, Python, aiogram, Polymarket CLOB / web3.py, PostgreSQL, ChangeNOW, TON (pytoniq), systemd + Docker for the frontend.
Solution	• Site: strategy grid, filtered leaderboard, trader card, Nuxt Content blog, i18n. • Workers: whale watcher / forwarder, copy engine, deposit watcher, auto-redeem, commission batcher, multichain wallets. • Deposit UX in the bot, ChangeNOW bridge, TON as an entry rail. • Separate support bot. Deploy: systemd for API/bots, Docker Compose for Nuxt.
Result	In production. Code is mine, the CEO runs the product.
Link	t.me/polyxcopy_bot